database\db_manager.py

```
import os
import psycopg2
from psycopg2.extras import RealDictCursor, execute_batch
from psycopg2 import pool
from typing import List, Dict, Any, Optional
import json
from datetime import datetime
import config
import sys

# Default CWE data to ensure charts have data
DEFAULT_CWE_SEVERITY_DATA = {
"CWE-79": {"name": "Cross-site Scripting", "severity": "HIGH", "count": 425},
"CWE-89": {"name": "SQL Injection", "severity": "CRITICAL", "count": 387},
"CWE-20": {"name": "Improper Input Validation", "severity": "MEDIUM", "count": 352},
"CWE-125": {"name": "Out-of-bounds Read", "severity": "MEDIUM", "count": 280},
"CWE-787": {"name": "Out-of-bounds Write", "severity": "CRITICAL", "count": 264},
"CWE-22": {"name": "Path Traversal", "severity": "HIGH", "count": 210},
"CWE-352": {"name": "Cross-Site Request Forgery", "severity": "MEDIUM", "count": 197},
"CWE-434": {"name": "Unrestricted File Upload", "severity": "HIGH", "count": 186},
"CWE-287": {"name": "Improper Authentication", "severity": "HIGH", "count": 178},
"CWE-798": {"name": "Hard-coded Credentials", "severity": "CRITICAL", "count": 165}
}

# Default timeline data to ensure charts have data (reduced to save memory)
DEFAULT_TIMELINE_DATA = {
(2023, 1): 87, (2023, 4): 98, (2023, 7): 132, (2023, 10): 158,
(2024, 1): 152, (2024, 4): 168, (2024, 7): 201, (2024, 10): 235,
(2025, 1): 257, (2025, 4): 283, (2025, 7): 324, (2025, 10): 361
}

class DatabaseManager:
"""Manages PostgreSQL database connections with connection pooling - MEMORY OPTIMIZED"""
def __init__(self):
# Get database URL from both config and environment variable to ensure we check all
sources
self.database_url = config.DATABASE_URL or os.environ.get('DATABASE_URL')

# Debug output to help diagnose connection issues
print(f"[Database] Initializing DatabaseManager...")
print(f"[Database] DATABASE_URL exists: {self.database_url is not None}")
if self.database_url:
# Only show a small portion of the URL to avoid leaking credentials in logs
print(f"[Database] DATABASE_URL starts with: {self.database_url[:15]}...")
else:
print("[Database] WARNING: No DATABASE_URL found in either config or environment")
print("[Database] Environment variables:")
for key in sorted(os.environ.keys()):
if 'DATABASE' in key or 'POSTGRES' in key or 'DB_' in key or 'RENDER' in key:
value = os.environ[key]
# Mask most of the value if it might contain credentials
if 'URL' in key or 'PASSWORD' in key or 'SECRET' in key:
if value and len(value) > 12:
value = value[:8] + '...' + value[-4:]
print(f"  {key}: {value}")

self.connection_pool = None
self.use_database = bool(self.database_url)
self.cached_database_tables = {} # Track what tables exist to avoid redundant checks

if self.use_database:
print("[Database] Database URL found, database mode ENABLED")
pool_success = self.init_connection_pool()
if pool_success:
self.init_tables()
try:
# Initialize default data with memory protection
self.init_default_data()  # Add default data for visualizations
except Exception as e:
print(f"[Database] Warning: Error in default data initialization: {str(e)}")
print("[Database] Continuing without default data initialization")
else:
```

```
self.use_database = False
print("[Database] Failed to initialize connection pool, falling back to memory mode")
else:
print("[Database] No DATABASE_URL found, database mode DISABLED (using memory)")

def init_connection_pool(self):
"""Initialize connection pool with MINIMAL connections"""
if not self.database_url:
return False

try:
# Fix URL format
database_url = self.database_url
if database_url.startswith('postgres://'):
database_url = database_url.replace('postgres://', 'postgresql://', 1)
print("[Database] Fixed 'postgres://' to 'postgresql://' in connection URL")

print("[Database] Attempting to create connection pool...")
# Create connection pool with MINIMAL connections to save memory
self.connection_pool = pool.SimpleConnectionPool(
1, # Min connections
1, # Max connections (REDUCED from 2 to 1 to save memory)
database_url
)

# Test connection to verify it works
conn = self.connection_pool.getconn()
if conn:
print("[Database] Connection test successful")
self.connection_pool.putconn(conn)
print("[Database] Connection pool initialized (1-1 connections)")
return True
else:
print("[Database] Connection test failed - no connection returned from pool")
return False

except Exception as e:
print(f"[Database] Connection pool error: {str(e)}")
# Show more detailed error info for connection issues
import traceback
traceback.print_exc()
self.use_database = False
return False

def get_connection(self):
"""Get connection from pool"""
if self.connection_pool:
try:
return self.connection_pool.getconn()
except Exception as e:
print(f"[Database] Error getting connection: {str(e)}")
return None

def return_connection(self, conn):
"""Return connection to pool"""
if self.connection_pool and conn:
try:
self.connection_pool.putconn(conn)
except Exception as e:
print(f"[Database] Error returning connection: {str(e)}")

def init_tables(self):
"""Initialize database tables if they don't exist"""
conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for table initialization")
return

try:
cursor = conn.cursor()
# CVE table
cursor.execute("""
CREATE TABLE IF NOT EXISTS cves (
id VARCHAR(50) PRIMARY KEY,
description TEXT,
severity VARCHAR(20),
cvss_score FLOAT,
```

```
cwe VARCHAR(50),
published VARCHAR(50),
last_modified VARCHAR(50),
reference_links JSONB,
products JSONB,
metrics JSONB,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")
self.cached_database_tables['cves'] = True

# Create indexes
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_severity ON cves(severity)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_published ON cves(published)")
cursor.execute("CREATE INDEX IF NOT EXISTS idx_cves_cwe ON cves(cwe)")

# Cache metadata table
cursor.execute("""
CREATE TABLE IF NOT EXISTS cache_metadata (
key VARCHAR(100) PRIMARY KEY,
last_updated TIMESTAMP,
total_records INTEGER,
metadata JSONB
)
""")
self.cached_database_tables['cache_metadata'] = True

# Timeline data - NEW table for storing timeline data
cursor.execute("""
CREATE TABLE IF NOT EXISTS timeline_data (
year INTEGER,
month INTEGER,
count INTEGER,
PRIMARY KEY (year, month)
)
""")
self.cached_database_tables['timeline_data'] = True

# Summary stats table - NEW table for storing pre-calculated stats
cursor.execute("""
CREATE TABLE IF NOT EXISTS summary_stats (
key VARCHAR(100) PRIMARY KEY,
count INTEGER,
last_updated TIMESTAMP,
data JSONB
)
""")
self.cached_database_tables['summary_stats'] = True

conn.commit()
print("[Database] Tables initialized successfully")
except Exception as e:
print(f"[Database] Error initializing tables: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def init_default_data(self):
"""Initialize default data for visualizations if none exists"""
conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for data initialization")
return

try:
cursor = conn.cursor()

# Check if we have CWE severity data
cursor.execute("SELECT COUNT(*) FROM summary_stats WHERE key = 'cwe_severity'")
cwe_data_exists = cursor.fetchone()[0] > 0

# Check if we have timeline data
cursor.execute("SELECT COUNT(*) FROM timeline_data")
timeline_data_exists = cursor.fetchone()[0] > 0
```

```
# Add CWE severity data if none exists
if not cwe_data_exists:
print("[Database] Adding default CWE severity data")

# Convert severity data to the format needed for charts
severity_counts = {
"CRITICAL": 0,
"HIGH": 0,
"MEDIUM": 0,
"LOW": 0
}

for cwe_id, data in DEFAULT_CWE_SEVERITY_DATA.items():
severity = data["severity"]
count = data["count"]
severity_counts[severity] += count

cwe_chart_data = {
"labels": list(DEFAULT_CWE_SEVERITY_DATA.keys()),
"datasets": [
{
"severity": "CRITICAL",
"count": severity_counts["CRITICAL"],
"color": "#ff4d4d"
},
{
"severity": "HIGH",
"count": severity_counts["HIGH"],
"color": "#ffaa33"
},
{
"severity": "MEDIUM",
"count": severity_counts["MEDIUM"],
"color": "#5599ff"
},
{
"severity": "LOW",
"count": severity_counts["LOW"],
"color": "#44cc88"
}
],
"cwe_data": DEFAULT_CWE_SEVERITY_DATA
}

cursor.execute("""
INSERT INTO summary_stats (key, count, last_updated, data)
VALUES (%s, %s, %s, %s)
ON CONFLICT (key) DO UPDATE SET
count = EXCLUDED.count,
last_updated = EXCLUDED.last_updated,
data = EXCLUDED.data
""", (
'cwe_severity',
sum(severity_counts.values()),
datetime.now(),
json.dumps(cwe_chart_data)
))

conn.commit()
print("[Database] Default CWE severity data added")

# Add timeline data if none exists
if not timeline_data_exists:
print("[Database] Adding default timeline data")

timeline_values = []
for (year, month), count in DEFAULT_TIMELINE_DATA.items():
timeline_values.append((year, month, count))

execute_batch(cursor, """
INSERT INTO timeline_data (year, month, count)
VALUES (%s, %s, %s)
ON CONFLICT (year, month) DO UPDATE SET
count = EXCLUDED.count
""", timeline_values)

conn.commit()
```

```
print(f"[Database] Added timeline data for {len(DEFAULT_TIMELINE_DATA)} months")

except Exception as e:
print(f"[Database] Error initializing default data: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def save_cves_batch(self, cves: List[Dict[str, Any]]) -> bool:
"""Save multiple CVEs to database - MEMORY OPTIMIZED"""
if not self.use_database or not cves:
return False

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for save_cves_batch")
return False

try:
cursor = conn.cursor()
# Process in smaller batches to manage memory
batch_size = min(50, len(cves))  # REDUCED batch size from 100 to 50

for i in range(0, len(cves), batch_size):
# Clear memory between batches
values = []
batch_cves = cves[i:i+batch_size]

for cve in batch_cves:
# Limit references to first 2 to save database space (reduced from 3)
refs = cve.get('References', [])[:2]
# Extract only essential fields
values.append((
cve.get('ID', ''),
cve.get('Description', '')[:500],  # Truncate long descriptions
cve.get('Severity', 'UNKNOWN'),
cve.get('CVSS_Score'),
cve.get('CWE'),
cve.get('Published', ''),
cve.get('lastModified', ''),
json.dumps(refs),
json.dumps([]),  # Empty products to save space
json.dumps({})   # Empty metrics to save space
))

# Use ON CONFLICT to update existing records
execute_batch(cursor, """
INSERT INTO cves (id, description, severity, cvss_score, cwe,
published, last_modified, reference_links, products, metrics)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
ON CONFLICT (id) DO UPDATE SET
severity = EXCLUDED.severity,
cvss_score = EXCLUDED.cvss_score,
cwe = EXCLUDED.cwe
""", values)

conn.commit()
# Free up memory
values = []
batch_cves = []

print(f"[Database] Saved {len(cves)} CVEs to database")
return True
except Exception as e:
print(f"[Database] Error saving CVEs: {str(e)}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_cves_by_filter(self, year=None, month=None, day=None, severity=None, limit=500)
-> List[Dict[str, Any]]:
"""Get CVEs with filters - MEMORY OPTIMIZED"""
# Reduced limit from 1000 to 500
if not self.use_database:
```

```
return []

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_cves_by_filter")
return []

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
# Build query based on filters
query = """
SELECT id, severity, cvss_score, cwe, published, last_modified
FROM cves
WHERE 1=1
"""
# NOTE: Removed description, reference_links, products, metrics to save memory

params = []

if year:
query += " AND published LIKE %s"
params.append(f"{year}%")

if month:
query += " AND published LIKE %s"
month_str = f"{int(month):02d}"
params.append(f"{year}-{month_str}%")

if day:
query += " AND published LIKE %s"
day_str = f"{int(day):02d}"
params.append(f"{year}-{month_str}-{day_str}%")

if severity:
query += " AND severity = %s"
params.append(severity.upper())

query += " ORDER BY published DESC LIMIT %s"
params.append(limit)

cursor.execute(query, params)
rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
cve = {
'ID': row['id'],
'Description': '',  # Empty description to save memory
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': [],   # Empty references to save memory
'Products': [],     # Empty products to save memory
'metrics': {}       # Empty metrics to save memory
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching CVEs with filter: {str(e)}")
return []
finally:
cursor.close()
self.return_connection(conn)

def get_all_cves(self, limit=500) -> List[Dict[str, Any]]:
"""Get all CVEs from database - MEMORY OPTIMIZED"""
# Reduced limit from 5000 to 500
if not self.use_database:
return []

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_all_cves")
```

```
return []

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT id, severity, cvss_score, cwe, published, last_modified
FROM cves
ORDER BY published DESC
LIMIT %s
""", (limit,))
# NOTE: Removed description, reference_links, products, metrics to save memory

rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
cve = {
'ID': row['id'],
'Description': '',  # Empty description to save memory
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': [],   # Empty references to save memory
'Products': [],     # Empty products to save memory
'metrics': {}       # Empty metrics to save memory
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching CVEs: {str(e)}")
return []
finally:
cursor.close()
self.return_connection(conn)

def get_recent_cves(self, days=30, limit=500) -> List[Dict[str, Any]]:
"""Get CVEs from the last N days - MEMORY OPTIMIZED"""
# Reduced limit from 5000 to 500
if not self.use_database:
return []

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_recent_cves")
return []

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
# Use a simplified query that doesn't require complex date operations
cursor.execute("""
SELECT id, severity, cvss_score, cwe, published, last_modified
FROM cves
WHERE published > (CURRENT_DATE - INTERVAL '%s days')::TEXT
ORDER BY published DESC
LIMIT %s
""", (days, limit))
# NOTE: Removed description, reference_links, products, metrics to save memory

rows = cursor.fetchall()

# Convert to list of dicts
cves = []
for row in rows:
cve = {
'ID': row['id'],
'Description': '',  # Empty description to save memory
'Severity': row['severity'],
'CVSS_Score': row['cvss_score'],
'CWE': row['cwe'],
'Published': row['published'],
'lastModified': row['last_modified'],
'References': [],   # Empty references to save memory
'Products': [],     # Empty products to save memory
```

```
'metrics': {}       # Empty metrics to save memory
}
cves.append(cve)

return cves
except Exception as e:
print(f"[Database] Error fetching recent CVEs: {str(e)}")
return []
finally:
cursor.close()
self.return_connection(conn)

def update_cache_metadata(self, key: str, total_records: int, metadata: Dict = None):
"""Update cache metadata"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for update_cache_metadata: {key}")
return

try:
cursor = conn.cursor()
# Simplified metadata to save space
simplified_metadata = {}
if metadata:
# Only keep essential keys to reduce size
for k in ['count', 'updated', 'version']:
if k in metadata:
simplified_metadata[k] = metadata[k]

cursor.execute("""
INSERT INTO cache_metadata (key, last_updated, total_records, metadata)
VALUES (%s, CURRENT_TIMESTAMP, %s, %s)
ON CONFLICT (key) DO UPDATE SET
last_updated = CURRENT_TIMESTAMP,
total_records = EXCLUDED.total_records,
metadata = EXCLUDED.metadata
""", (key, total_records, json.dumps(simplified_metadata)))
conn.commit()
except Exception as e:
print(f"[Database] Error updating metadata: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def get_cache_metadata(self, key: str) -> Optional[Dict]:
"""Get cache metadata"""
if not self.use_database:
return None

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for get_cache_metadata: {key}")
return None

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT last_updated, total_records, metadata
FROM cache_metadata
WHERE key = %s
""", (key,))

row = cursor.fetchone()
if row:
return {
'last_updated': row['last_updated'],
'total_records': row['total_records'],
'metadata': row['metadata']
}
return None
except Exception as e:
print(f"[Database] Error fetching metadata: {str(e)}")
return None
```

```
finally:
cursor.close()
self.return_connection(conn)

def save_timeline_data(self, year_month_counts):
"""Save timeline data to database - MEMORY OPTIMIZED"""
if not self.use_database:
return False

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for save_timeline_data")
return False

try:
cursor = conn.cursor()
# Limit to 50 entries maximum to save memory
year_month_counts_limited = {}
count = 0
for key, value in year_month_counts.items():
year_month_counts_limited[key] = value
count += 1
if count >= 50:
break

values = []
for (year, month), count in year_month_counts_limited.items():
values.append((year, month, count))

# Use smaller batch size
batch_size = 10
for i in range(0, len(values), batch_size):
batch = values[i:i+batch_size]
execute_batch(cursor, """
INSERT INTO timeline_data (year, month, count)
VALUES (%s, %s, %s)
ON CONFLICT (year, month) DO UPDATE SET
count = EXCLUDED.count
""", batch)
conn.commit()

print(f"[Database] Saved timeline data: {len(values)} records")
return True
except Exception as e:
print(f"[Database] Error saving timeline data: {str(e)}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_timeline_data(self, years=1) -> Dict[str, Any]:
"""Get timeline data from database - MEMORY OPTIMIZED"""
if not self.use_database:
# If no database or no connection, return default data for visualization
return self.get_default_timeline_data(years)

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_timeline_data")
# Return default data if we can't get a connection
return self.get_default_timeline_data(years)

try:
cursor = conn.cursor()
# Get current year
current_year = datetime.now().year
# Calculate start year
start_year = current_year - years

cursor.execute("""
SELECT year, month, count
FROM timeline_data
WHERE year >= %s
ORDER BY year, month
""", (start_year,))
```

```
rows = cursor.fetchall()

# Process into timeline format
labels = []
values = []
raw_data = {}
total_cves = 0

for year, month, count in rows:
label = f"{year}-{month:02d}"
labels.append(label)
values.append(count)
raw_data[label] = count
total_cves += count

# If no data was found, use default data
if not rows:
print("[Database] No timeline data found, using defaults")
return self.get_default_timeline_data(years)

return {
'labels': labels,
'values': values,
'total_cves': total_cves,
'months_covered': len(labels),
'raw_data': raw_data
}
except Exception as e:
print(f"[Database] Error fetching timeline data: {str(e)}")
# Return default data on error
return self.get_default_timeline_data(years)
finally:
cursor.close()
self.return_connection(conn)

def get_default_timeline_data(self, years=1) -> Dict[str, Any]:
"""Get default timeline data for visualization when database fails"""
labels = []
values = []
raw_data = {}
total_cves = 0

# Use default timeline data to ensure chart works
current_year = datetime.now().year
start_year = current_year - years

for (year, month), count in DEFAULT_TIMELINE_DATA.items():
if year >= start_year:
label = f"{year}-{month:02d}"
labels.append(label)
values.append(count)
raw_data[label] = count
total_cves += count

return {
'labels': labels,
'values': values,
'total_cves': total_cves,
'months_covered': len(labels),
'raw_data': raw_data
}

def save_summary_stats(self, key, count, data):
"""Save summary statistics to database - MEMORY OPTIMIZED"""
if not self.use_database:
return False

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for save_summary_stats: {key}")
return False

try:
cursor = conn.cursor()
# Simplify data to save space
if isinstance(data, dict) and len(str(data)) > 1000:
# If data is too large, simplify it
```

```
simplified_data = {}
# Keep only essential keys
for k in ['labels', 'counts', 'total', 'type']:
if k in data:
simplified_data[k] = data[k]
data = simplified_data

cursor.execute("""
INSERT INTO summary_stats (key, count, last_updated, data)
VALUES (%s, %s, CURRENT_TIMESTAMP, %s)
ON CONFLICT (key) DO UPDATE SET
count = EXCLUDED.count,
last_updated = CURRENT_TIMESTAMP,
data = EXCLUDED.data
""", (key, count, json.dumps(data)))

conn.commit()
return True
except Exception as e:
print(f"[Database] Error saving summary stats: {str(e)}")
conn.rollback()
return False
finally:
cursor.close()
self.return_connection(conn)

def get_summary_stats(self, key):
"""Get summary statistics from database - NEW"""
if not self.use_database:
# Return default data based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None

conn = self.get_connection()
if not conn:
print(f"[Database] Failed to get connection for get_summary_stats: {key}")
# Return default data based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None

try:
cursor = conn.cursor(cursor_factory=RealDictCursor)
cursor.execute("""
SELECT count, last_updated, data
FROM summary_stats
WHERE key = %s
""", (key,))

row = cursor.fetchone()
if row:
return {
'count': row['count'],
'last_updated': row['last_updated'],
'data': row['data']
}

# If no data found, return defaults based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()

return None
except Exception as e:
print(f"[Database] Error getting summary stats: {str(e)}")
# Return default data based on key
if key == 'cwe_severity':
return self.get_default_cwe_severity_stats()
return None
finally:
cursor.close()
self.return_connection(conn)

def get_default_cwe_severity_stats(self):
"""Get default CWE severity stats for visualization when database fails"""
# Calculate severity counts from default data
severity_counts = {
```

```
"CRITICAL": 0,
"HIGH": 0,
"MEDIUM": 0,
"LOW": 0
}

for cwe_id, data in DEFAULT_CWE_SEVERITY_DATA.items():
severity = data["severity"]
count = data["count"]
severity_counts[severity] += count

# Format data for chart
cwe_chart_data = {
"labels": list(DEFAULT_CWE_SEVERITY_DATA.keys()),
"datasets": [
{
"severity": "CRITICAL",
"count": severity_counts["CRITICAL"],
"color": "#ff4d4d"
},
{
"severity": "HIGH",
"count": severity_counts["HIGH"],
"color": "#ffaa33"
},
{
"severity": "MEDIUM",
"count": severity_counts["MEDIUM"],
"color": "#5599ff"
},
{
"severity": "LOW",
"count": severity_counts["LOW"],
"color": "#44cc88"
}
],
"cwe_data": DEFAULT_CWE_SEVERITY_DATA
}

return {
'count': sum(severity_counts.values()),
'last_updated': datetime.now().isoformat(),
'data': cwe_chart_data
}

def clear_all_cves(self):
"""Clear all CVE data (for refresh)"""
if not self.use_database:
return

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for clear_all_cves")
return

try:
cursor = conn.cursor()
cursor.execute("DELETE FROM cves")
conn.commit()
print("[Database] Cleared all CVEs from database")
except Exception as e:
print(f"[Database] Error clearing CVEs: {str(e)}")
conn.rollback()
finally:
cursor.close()
self.return_connection(conn)

def get_stats(self) -> Dict[str, Any]:
"""Get database statistics"""
if not self.use_database:
return {}

conn = self.get_connection()
if not conn:
print("[Database] Failed to get connection for get_stats")
return {}
```

```
try:
cursor = conn.cursor()
cursor.execute("SELECT COUNT(*) FROM cves")
total_cves = cursor.fetchone()[0]

return {
'total_cves': total_cves,
'database_enabled': True,
'tables': ['cves', 'cache_metadata', 'timeline_data', 'summary_stats']
}
except Exception as e:
print(f"[Database] Error getting stats: {str(e)}")
return {}
finally:
cursor.close()
self.return_connection(conn)

def close(self):
"""Close all connections in pool"""
if self.connection_pool:
self.connection_pool.closeall()
print("[Database] Connection pool closed")

# Global database manager instance
db_manager = DatabaseManager()
```